DOCUMENTAZIONE DI PROGETTO

StudyCoach

Relazione tecnica di progetto

Architettura, gestione dei documenti
e percorso di studio assistito

RELAZIONE TECNICA

19 ottobre 2026

Indice

1. Scopo e funzionalità 6

2. Architettura generale del sistema 6

Percorsi principali 7

3. Requisiti e avvio 8

Configurazione 8

4. Persistenza e modello dati 9

5. Flusso dei PDF e delle fonti 10

6. Generazione del quiz 11

7. Sessione di studio e tutor 11

8. Appunti e chat dei dubbi 13

9. Interfaccia e API 13

10. Autenticazione e sicurezza 14

11. Osservabilità e gestione errori 14

12. Test e valutazione 15

13. Procedure operative e recupero 15

14. Limiti tecnici e punti di estensione 16

15. Organizzazione della manutenzione 17

16. Analisi dettagliata dei livelli 18

16.1 Processo e ordine di inizializzazione 18

16.2 Confine tra browser, API e servizi 18

16.3 Moduli di dominio 18

16.4 Dipendenze e ruolo 19

17. Contratti HTTP e flussi completi 20

17.1 Catalogo delle API JSON 20

17.2 Registrazione, login e accesso alle risorse 21

17.3 Upload di un PDF: sequenza e compensazioni 22

17.4 Dalla selezione delle fonti al quiz 22

17.5 Macchina a stati di una domanda 23

17.6 Sessione, tempo, adattamento e report 24

17.7 Chat dei dubbi e appunti 24

18. Algoritmi e dati persistiti 26

18.1 Relazioni e campi di stato 26

18.2 Chunking e identità della fonte 27

18.3 Ricerca e filtro delle fonti 27

18.4 Copertura e controllo del quiz 28

18.5 Calcolo del tempo e dei voti 28

19. Errori, consistenza e recupero 29

19.1 Confini transazionali 29

19.2 Matrice di errori osservabili 29

19.3 Backup e ripristino 30

19.4 Diagnosi della configurazione LLM 30

20. Qualità, verifica e limiti delle evidenze 31

20.1 Test automatici 31

20.2 Validazione real LLM 31

20.3 Criteri di verifica 31

20.4 Ambito della documentazione 31

21. Divisione dei contributi: agenti applicativi 32

21.1 Riepilogo per sviluppatore 32

22. Utilizzo di intelligenza artificiale durante lo sviluppo 33

22.1 GitHub Copilot 33

22.2 Claude 33

Presentazione della relazione

Questa relazione descrive StudyCoach attraverso le funzioni offerte allo studente e i componenti che le realizzano. Il percorso di lettura parte dagli obiettivi, introduce architettura e persistenza, segue il trattamento dei PDF e approfondisce quiz, tutor, appunti e chat. La seconda parte analizza i contratti HTTP, gli algoritmi, gli stati persistiti e le procedure di recupero.

Le spiegazioni si riferiscono all'implementazione descritta nella documentazione tecnica del progetto. I percorsi di file sono relativi alla cartella principale del repository. I nomi delle funzioni e dei campi sono mantenuti per consentire il confronto fra relazione e codice. Le istruzioni complete di installazione e avvio sono raccolte nel file AVVIO.md del progetto.

I diagrammi riproducono le definizioni originali. Il documento distingue i controlli eseguiti dal codice dalle proprietà che dipendono dai modelli linguistici e dalle condizioni dell'ambiente. Questa distinzione permette di interpretare correttamente valutazioni, risultati storici e verifiche automatiche.

1. Scopo e funzionalità

Il progetto affronta un problema frequente nella preparazione di un esame: disporre di molti documenti non significa avere già un percorso di studio organizzato. Lo studente deve individuare i concetti, verificare ciò che ha compreso e riconoscere gli argomenti sui quali tornare. StudyCoach riunisce queste attività in un ambiente che conserva il collegamento fra il materiale caricato, le domande proposte e i risultati delle sessioni.

Il punto di partenza è l'esame, che costituisce il contenitore logico dei documenti e delle attività. Questa scelta permette di separare materie diverse e di riutilizzare lo stesso materiale in più sessioni senza doverlo caricare ogni volta. La selezione delle fonti restringe ulteriormente il lavoro a una parte del programma. La sessione, invece, rappresenta un'attività delimitata nel tempo: raccoglie un quiz, i turni di risposta, le valutazioni e il report conclusivo.

L'approccio didattico adottato prevede un passaggio intermedio fra la prima risposta e il giudizio finale. La domanda guida invita lo studente a rielaborare il ragionamento, rendendo osservabile anche l'evoluzione della risposta. Il sistema svolge quindi una funzione di supporto all'esercitazione; il voto prodotto è un dato del percorso applicativo e deve essere interpretato insieme alla motivazione e ai limiti della valutazione automatica.

StudyCoach è un'applicazione di studio per esami universitari. Uno studente crea un account e uno o più esami, carica PDF, sceglie le fonti da usare e genera un quiz basato sui passaggi indicizzati. Durante la sessione risponde a una domanda, riceve una domanda guida socratica, risponde di nuovo, ottiene una valutazione e può contestarla. L'app sceglie le domande successive usando gli argomenti deboli. Produce report e appunti PDF. Una chat separata aiuta ad affrontare dubbi sul materiale dell'esame.

La generazione e la valutazione usano modelli linguistici remoti; gli embedding dei PDF e l'OCR sono locali. Le risposte del modello sono sottoposte a controlli applicativi, ma non costituiscono una valutazione docente garantita.

2. Architettura generale del sistema

L'architettura può essere letta seguendo una richiesta dello studente. Il browser raccoglie un'azione, come il caricamento di un documento o l'invio di una risposta, e la trasmette all'applicazione FastAPI. Il livello applicativo controlla identità, proprietà delle risorse e validità dei dati; successivamente richiama il servizio che realizza l'operazione. La risposta restituita al browser può essere una pagina HTML, un oggetto JSON, un flusso di aggiornamenti oppure un file da scaricare.

La scelta di un monolite mantiene nello stesso progetto il livello web e la logica di dominio. Questo non elimina la separazione delle responsabilità: i router gestiscono il contratto di accesso, gli agenti applicativi svolgono i compiti di elaborazione e i componenti di persistenza conservano dati diversi. La distinzione è particolarmente rilevante perché un file PDF, una sessione di studio e un vettore semantico richiedono modalità di archiviazione differenti.

Nel diagramma seguente le frecce descrivono le principali dipendenze. I provider linguistici sono servizi esterni raggiunti dagli agenti; l'estrazione e la rappresentazione vettoriale dei documenti avvengono invece mediante componenti locali. Il database relazionale e l'indice vettoriale collaborano, ma mantengono ruoli distinti.

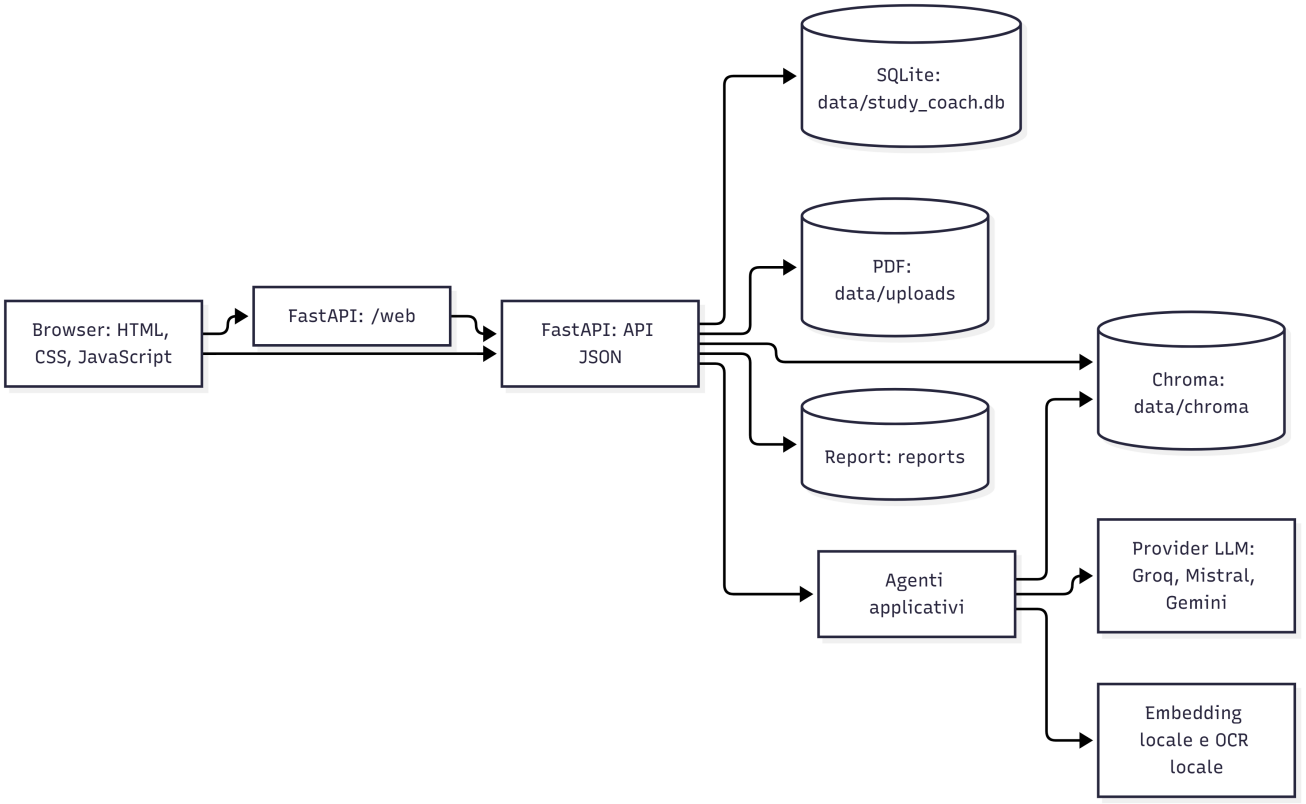


Figura 1. Architettura generale e dipendenze principali.

È un monolite Python. `backend/api/main.py` costruisce l'app FastAPI, monta i router API e il router web, serve `frontend/static` e inizializza il DB all'avvio. `frontend/web.py` rende i template Jinja e coordina i form del browser richiamando i servizi applicativi. Non sono necessari Node, npm o un server frontend distinto. Il browser accede normalmente allo stesso host dell'API; `CORS_ORIGINS` serve per eventuali client esterni.

Percorsi principali

Percorso	Responsabilità
`backend/api/main.py`	App, lifespan, CORS, middleware, router, health check.
`backend/api/auth.py`, `authorization.py`	Registrazione, login, logout e controllo del proprietario delle risorse.
`backend/api/exams.py`, `frontend_extras.py`	Esami, upload, fonti, verifica, reindicizzazione, appunti, dubbi.
`backend/api/sessions.py`, `session_extras.py`	Quiz, tutor, valutazione, sessioni, report, ripresa, appunti da sessione.
`frontend/web.py`, `frontend/templates/`, `frontend/static/`	Interfaccia HTML/Jinja, stili, form, avanzamento operazioni e timer.
`backend/db/models.py`	Modelli SQLAlchemy, connessione SQLite, creazione e aggiornamento compatibile delle colonne.
`backend/agents/ingestor.py`	Lettura PDF, OCR, chunk, embedding, Chroma, retrieval e verifica di coerenza.
`backend/agents/quiz_generator.py`	Generazione di quiz con controllo delle citazioni.

Percorso	Responsabilità
`backend/agents/socratic_tutor.py`	Loop socratico, valutazione, contestazione e tutor dei dubbi.
`backend/agents/notes_generator.py`	Appunti, verifica rispetto alle fonti e impaginazione PDF.
`backend/agents/adaptive_engine.py`	Priorità delle domande e statistiche per argomento.
`backend/agents/report_generator.py`	Report Markdown/PDF e gestione dei file generati.
`backend/agents/model_router.py`, `llm_errors.py`	Scelta provider/modello, errori, timeout e tentativi.
`backend/agents/prompts/`	Versioni testuali dei prompt per quiz, tutor e appunti.
`backend/models/schemas.py`	Schemi Pydantic di richiesta e risposta.
`eval/`	Casi di validazione real LLM, materiali di riferimento e risultati storici.
`test/`	Test unitari e di integrazione.

3. Requisiti e avvio

I requisiti di esecuzione comprendono sia l'ambiente software sia le risorse necessarie ai dati. Python esegue il backend e i componenti di elaborazione; uv gestisce l'installazione delle dipendenze. Lo spazio su disco non riguarda soltanto i PDF originali, ma anche i vettori, i file prodotti e la cache dei modelli locali. La prima operazione su un documento può quindi richiedere più tempo delle successive, poiché alcuni componenti vengono inizializzati al primo utilizzo.

La configurazione è separata dal codice per rendere riproducibili le scelte operative senza includere credenziali nei file versionati. Le chiavi autorizzano l'accesso ai servizi remoti, mentre i valori di configurazione stabiliscono quale servizio usare, quanto attendere e quanti tentativi effettuare. L'ordine di precedenza dei valori consente a un'installazione di personalizzare i parametri senza modificare i valori predefiniti del progetto.

StudyCoach richiede Python **3.13 o successivo**, `uv`, spazio per database, PDF, indice Chroma e cache del modello di embedding. Il primo download del modello e le richieste ai provider LLM richiedono una connessione di rete. La procedura di installazione, la configurazione delle chiavi e l'avvio sono descritti esclusivamente in AVVIO.md.

L'interfaccia è servita dalla stessa applicazione FastAPI. La documentazione interattiva delle API è disponibile su `/docs` e `/redoc`; `/health` restituisce lo stato basilare del processo. All'avvio `init_db()` crea le tabelle mancanti e aggiunge alcune colonne necessarie ai database esistenti.

Configurazione

`backend/__init__.py` carica prima `.env` senza sovrascrivere l'ambiente di processo, poi applica i valori di `config/app.toml` con `os.environ.setdefault`. **Precedenza:** variabili già presenti nel processo, valori di `.env`, valori di `config/app.toml`. `.env.example` contiene soltanto `GROQ_API_KEY`, `MISTRAL_API_KEY` e `GEMINI_API_KEY`. `model_router.py` accetta anche `GOOGLE_API_KEY` come alternativa per Google. Le chiavi reali restano esterne al repository.

Gruppo	Impostazioni principali	Uso
Modelli	`TUTOR_PROVIDER/MODEL`, `QUIZ_PROVIDER/MODEL`, `QUIZ_FALLBACK_PROVIDER/MODEL`, `NOTES_PROVIDER/MODEL`, `NOTES_FALLBACK_PROVIDER/MODEL`	Scelta per funzionalità. I valori correnti sono in `config/app.toml`.
Chiamate LLM	`LLM_MAX_ATTEMPTS`, `LLM_TIMEOUT_SECONDS`, `LLM_RETRY_BASE_SECONDS`	Tentativi, timeout e attesa.
PDF/RAG	`MAX_UPLOAD_BYTES`, `RAG_CHUNK_WORDS`, `RAG_CHUNK_OVERLAP_WORDS`, `RETRIEVAL_MAX_DISTANCE`	Limiti upload, suddivisione e filtro opzionale della distanza.
Quiz	`QUIZ_MAX_CONTEXT_CHARS`, `QUIZ_MAX_BATCHES`, `QUIZ_MAX_INVALID_ATTEMPTS`, `QUIZ_MAX_RECOVERY_CALLS`	Limiti della generazione.
Appunti	`NOTES_MAX_*`	Tentativi, contesto, chunk e token della generazione/verifica.
HTTP	`CORS_ORIGINS`	Origini aggiuntive separate da virgola.

Cambiare dimensione o sovrapposizione dei chunk richiede reindicizzare le fonti. L'identificatore `LocalEmbeddingFunction.name()` è mantenuto per compatibilità Chroma; il modello effettivo è registrato nei metadati della collection. La sua stabilità preserva la compatibilità anche quando cambia la configurazione.

4. Persistenza e modello dati

Il modello di persistenza distingue dati amministrativi, stato dell'apprendimento e contenuti documentali. Le informazioni che richiedono relazioni esplicite, come l'appartenenza di un esame a uno studente, sono conservate in SQLite. I documenti originali rimangono nel filesystem, mentre Chroma conserva i passaggi di testo e i vettori necessari al recupero semantico. I report sono prodotti derivati, costruiti a partire dalle informazioni registrate nella sessione.

Questa organizzazione permette di ricostruire il significato di ogni attività: una sessione appartiene a un esame, l'esame appartiene a uno studente e le fonti selezionate identificano i materiali effettivamente utilizzati. La conservazione dello stato del turno rende inoltre possibile distinguere una risposta ancora in lavorazione da un tentativo già chiuso. Tale distinzione evita di usare un risultato parziale come se fosse una valutazione definitiva.

Il DB è SQLite in `data/study_coach.db`. SQLAlchemy definisce:

Tabella	Contenuto essenziale
`students`	Username, email, hash password, hash del token di sessione e scadenza.
`login_attempts`	Contatori e finestra temporale del limite sui tentativi di accesso.
`exams`	Esami appartenenti a uno studente.
`materials`	PDF caricati: percorso, nome, SHA-256, numero pagine, pagine con testo/scartate, numero chunk.

Tabella	Contenuto essenziale
`sessions`	Esame/studente, stato, tempi, voto, quiz JSON, turno corrente JSON, log delle risposte JSON, percorso del report.
`doubt_threads`	Conversazioni di dubbi raggruppate per esame.
`doubt_conversations`	Dubbio, domanda guida, risposta studente e risultato.

Le relazioni principali sono `Student -> Exam -> Material/Session/DoubtThread/DoubtConversation`. Le foreign key SQLite sono attivate sulla connessione. L'indice vettoriale è separato dal DB relazionale e persiste in `data/chroma`, con una collection `student_<id>_exam_<id>`. I file sorgente risiedono in `data/uploads`; i report in `reports/`. Questi percorsi sono esclusi da Git. Il salvataggio complessivo richiede quindi un backup coerente di **SQLite, PDF, Chroma e report**; copiare soltanto il DB non basta.

`alembic/env.py` collega i metadati SQLAlchemy, ma il progetto non contiene revisioni Alembic. L'evoluzione attuale delle tabelle è gestita soprattutto dalle funzioni `_ensure_*` di `backend/db/models.py` eseguite da `init_db()`; questa soluzione richiede attenzione nelle distribuzioni e nei ripristini complessi.

5. Flusso dei PDF e delle fonti

Un PDF non viene inviato direttamente al modello come un unico documento indistinto. Il flusso di acquisizione lo trasforma in unità testuali più piccole, ciascuna associata alla provenienza. L'estrazione per pagina conserva un riferimento utile per citazioni e verifiche; l'OCR interviene quando il testo non è disponibile nel contenuto estraibile della pagina, come può accadere nei documenti ottenuti da scansione.

La suddivisione in chunk serve a rendere il materiale utilizzabile dal recupero e a rispettare i limiti dei modelli. La sovrapposizione mantiene una parte del contesto tra due frammenti consecutivi, riducendo il rischio che una spiegazione venga separata proprio nel punto che ne chiarisce il significato. L'embedding converte poi ciascun frammento in una rappresentazione numerica che permette il confronto con la domanda dello studente.

La provenienza resta necessaria anche dopo questa trasformazione. Hash, pagina e indice del frammento permettono di riconoscere il documento, evitare duplicazioni e verificare che il contenuto registrato corrisponda ancora al file originale. L'intera catena precede la generazione: se il materiale risulta incoerente, le operazioni che dipendono da esso devono essere fermate o recuperate.

1. L'utente carica un PDF sotto un esame. Gli endpoint controllano proprietà dell'esame, estensione/dimensione e duplicati; il limite predefinito è 25 MiB (`MAX_UPLOAD_BYTES`).
2. `ingest_pdf()` calcola SHA-256 dei byte, apre il PDF con `pypdf`, estrae testo per pagina e usa OCR locale (`pypdfium2` e `rapidocr`) per le pagine senza testo estraibile.
3. Il testo viene diviso in chunk con sovrapposizione. I chunk troppo brevi sono esclusi. Ogni record Chroma contiene testo, studente, esame, pagina, nome fonte e hash del file. L'ID contiene hash, pagina e indice del chunk.
4. Gli embedding sono calcolati localmente con `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` e inseriti nella collection dell'esame. Il modello viene caricato al primo utilizzo; il primo download può essere lento.
5. L'ingestione verifica gli ID effettivamente salvati e prova a rimuovere inserimenti parziali in caso di errore. I metadati del materiale vengono registrati in SQLite.

6. Prima di alcune operazioni `verify_exam_material()` confronta file presenti, hash, conteggio chunk in DB e Chroma e segnala anche chunk orfani quando verifica tutto l'esame. Il reindex riparte dai PDF conservati su disco e tenta di ripristinare i vecchi record Chroma se fallisce.

`material_ids` limita l'elaborazione ai materiali scelti. Il retrieval traduce gli ID in filtri Chroma su `file_hash`, con fallback per vecchi record basato sul nome della fonte. La selezione viene salvata in `sessions.quiz_json["material_ids"]` e ripresa dal tutor e dagli appunti generati dal log della sessione. L'isolamento per esame deriva sia dalla collection distinta sia dai controlli di proprietà negli endpoint.

`search_passages()` restituisce testo, fonte, pagina, ID e distanza per la ricerca semantica; `retrieve_passages()` legge i passaggi indicizzati in ordine di documento. `RETRIEVAL_MAX_DISTANCE` può filtrare i risultati della ricerca. Una ricerca vettoriale resta una misura di somiglianza, non una prova automatica di correttezza della risposta.

6. Generazione del quiz

La generazione del quiz comprende una fase di preparazione del contesto, una richiesta al modello e una fase di accettazione applicativa. Il modello propone le domande, ma il programma decide quali risultati possono essere utilizzati. La distinzione è necessaria perché una risposta formalmente ben scritta potrebbe contenere una citazione inesistente, una struttura errata oppure una domanda già prodotta.

Il controllo della citazione lega ogni proposta a un passaggio realmente disponibile. Questo controllo fornisce un riscontro sulla provenienza testuale, mentre la validità didattica resta una proprietà più ampia: una citazione presente non dimostra da sola che la domanda sia chiara o che la difficoltà sia appropriata. La richiesta del numero esatto di domande evita inoltre che un fallimento parziale venga presentato allo studente come un quiz completo.

`POST /sessions/{session_id}/quiz` verifica i materiali e chiama `generate_quiz()`. Il generatore raccoglie passaggi dalle fonti selezionate, costruisce gruppi con un limite di contesto e richiede un output strutturato al modello. Valida schema e citazione `source_quote` contro il testo effettivamente indicizzato, elimina domande duplicate e prova un recupero limitato se alcuni gruppi falliscono. Il provider principale del quiz e l'eventuale fallback sono configurabili. Se non ottiene il numero richiesto di domande verificabili, l'endpoint risponde con errore e non salva un quiz incompleto come valido.

Il quiz riuscito viene serializzato in `sessions.quiz_json` insieme agli ID delle fonti selezionate. Le domande hanno ID, testo, difficoltà (`base`, `intermedia`, `avanzata`) e `topic_hint`. I prompt sono versionati in `backend/agents/prompts/`; la risposta API include `prompt_version`.

7. Sessione di studio e tutor

La sessione è il contesto persistente nel quale si svolge l'esercitazione. Il suo stato descrive se lo studente sta lavorando, ha interrotto l'attività oppure ha concluso il percorso con un report. All'interno della sessione esiste un secondo livello di stato, relativo alla singola domanda. I due livelli sono collegati, ma rispondono a esigenze diverse: il primo gestisce durata e continuità, il secondo regola l'ordine delle interazioni con il tutor.

La persistenza di quiz, turno corrente e log consente al server di ricostruire il percorso senza affidarsi alla memoria della pagina aperta nel browser. Un aggiornamento della pagina non dovrebbe quindi trasformare una risposta parziale in una nuova domanda. Per lo stesso motivo, il tempo attivo viene calcolato usando dati registrati dal server, mentre il timer visibile rappresenta tali dati per lo studente.

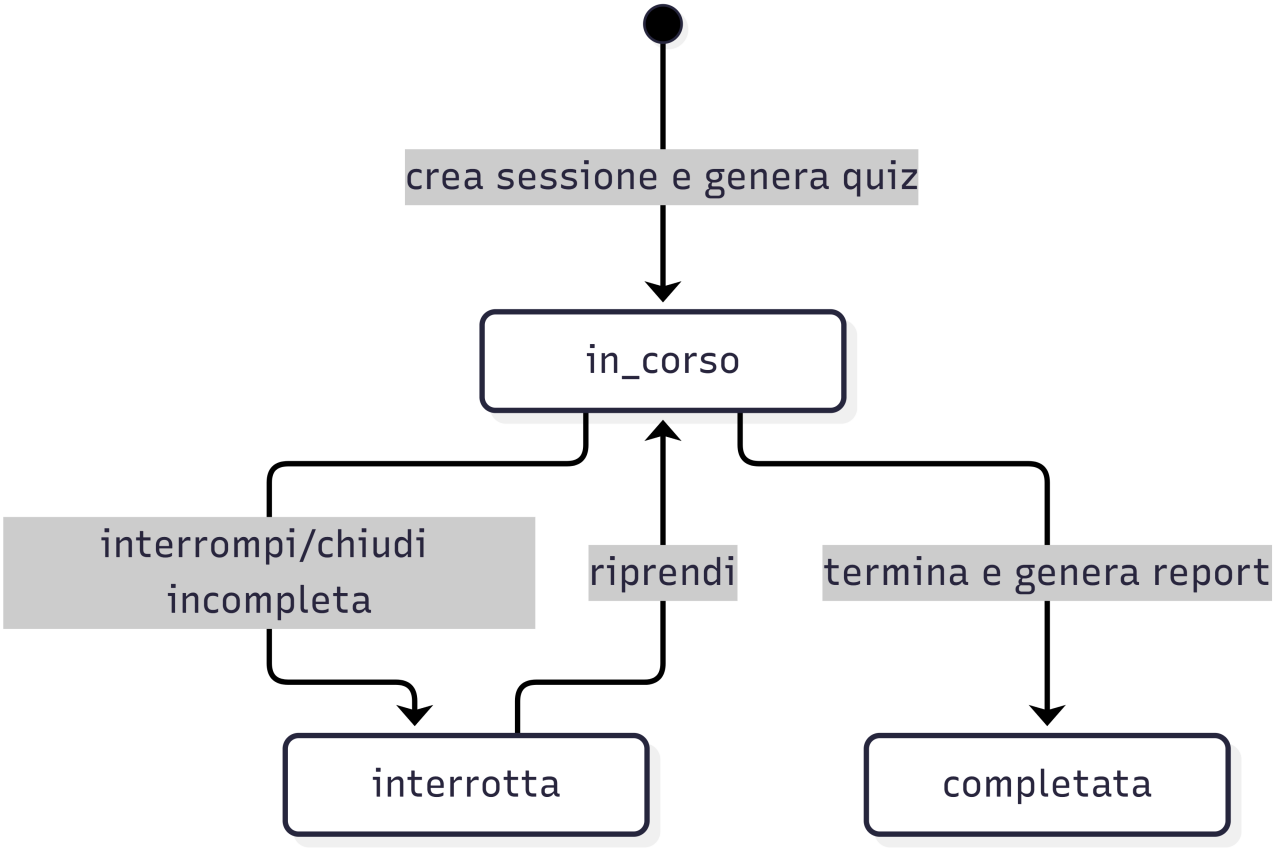


Figura 2. Ciclo di vita della sessione di studio.

La sessione salva `started_at`, `active_started_at` e `duration_seconds`. `backend/session_clock.py` accumula il tempo attivo durante pausa, ripresa e completamento; il browser mostra il timer con `frontend/static/session-timer.js`. `current_turn_json` permette di ricostruire il turno del tutor dopo una richiesta o un riavvio; `qa_log` conserva i tentativi già chiusi.

Per una domanda il tutor carica passaggi RAG delle fonti selezionate, riceve la prima risposta, formula una domanda guida e valuta la seconda risposta su scala 0–10. Restituisce motivazione, punti corretti, lacune e indicazione del miglioramento. Lo studente può inviare una contestazione motivata prima della chiusura della domanda. Lo stato del turno è serializzato tramite `SocraticTutor.to_json()`; il server ricostruisce il tutor da `current_turn_json` quando necessario. I moduli `quality_controls.py` ed `evaluation_validator.py` controllano struttura, coerenza e casi di risposte prive di contenuto.

`AdaptiveEngine` registra gli esiti affidabili e sceglie prima domande di argomenti deboli, poi argomenti non ancora affrontati, poi la prima domanda rimasta. I voti numerici con `reliable=False` possono comparire nella media/report storico, mentre la selezione adattiva li esclude. La distinzione evita che una valutazione dichiarata inaffidabile orienti il percorso successivo.

Alla chiusura, il report contiene media, numero di domande, dettaglio per argomento e sintesi. `report_generator.py` crea Markdown e PDF in `reports/` e salva il percorso del PDF nella sessione. Gli endpoint permettono download e rigenerazione del report a partire dal log, quando è disponibile un voto finale. Il PDF è una fotografia dei dati di sessione, non la fonte primaria: i dati primari sono nel DB.

8. Appunti e chat dei dubbi

Gli appunti e la chat dei dubbi rispondono a due esigenze complementari. Gli appunti organizzano materiale già disponibile in una forma adatta al ripasso; la chat parte invece da una difficoltà espressa dallo studente e costruisce un'interazione su quel punto. Entrambe le funzionalità riutilizzano l'indice dei documenti, ma differiscono per contenuto iniziale, stato conservato e risultato finale.

La generazione degli appunti da fonti selezionate richiede copertura del materiale e controllo della fedeltà; gli appunti ricavati da una sessione tengono conto anche degli argomenti affrontati e delle risposte registrate. Nella chat, il thread raccoglie una conversazione relativa all'esame e permette di ritrovarne la cronologia. La distinzione evita di confondere una discussione libera su un dubbio con il completamento di una domanda del quiz.

Gli appunti d'esame si possono generare da una o più fonti come PDF (`POST /exams/{exam_id}/notes/pdf`). Il generatore legge i passaggi indicizzati e divide il lavoro in parti. Per ogni parte, un Team Agno in modalità `coordinate` comprende un leader e un agente incaricato della stesura. Dopo il Team, un agente separato controlla la fedeltà alla fonte; i controlli applicativi verificano anche la struttura prima dell'impaginazione con `fpdf2` e font Unicode inclusi nel repository. Il renderer PDF riconosce anche le tabelle Markdown e le disegna come celle con testo a capo, bordi e intestazione ripetuta dopo i salti di pagina. La UI mostra l'avanzamento delle fasi e propone il download solo dopo la generazione completa. Esiste anche una risposta testuale per gli appunti d'esame e un endpoint che produce appunti dal `qa_log` di una sessione, arricchiti con retrieval limitato alle fonti del quiz. Quest'ultimo percorso usa ancora il singolo agente di scrittura.

La chat dei dubbi è distinta dalla sessione quiz. Un dubbio viene salvato in un thread per esame; `DoubtTutor` cerca il materiale pertinente e formula una domanda guida. La risposta dello studente viene poi valutata e persistita in `doubt_conversations`. L'interfaccia espone la cronologia. Un dubbio aperto nello stesso thread va risolto o scartato prima di iniziarne un altro.

9. Interfaccia e API

L'interfaccia web traduce le operazioni del dominio in pagine e form utilizzabili dallo studente. La libreria degli esami permette l'organizzazione iniziale; il dettaglio dell'esame rende disponibili i documenti e le attività; la pagina di studio accompagna i turni del tutor. La cronologia rende consultabili attività e risultati già registrati.

Le API JSON offrono accesso agli stessi ambiti funzionali attraverso richieste strutturate. La validazione dei payload e i controlli di autorizzazione avvengono sul server, indipendentemente dai vincoli mostrati dall'interfaccia. Per le operazioni lunghe, l'avanzamento trasmesso durante la richiesta permette di distinguere elaborazione in corso, completamento ed errore, senza presentare un risultato prima che sia effettivamente disponibile.

La UI è server side: template Jinja in `frontend/templates/`, CSS e JS in `frontend/static/`. Le pagine coprono autenticazione, libreria esami, dettaglio esame/fonti, studio, chat e cronologia. Alcune operazioni lunghe usano risposte streaming per aggiornare l'avanzamento nel browser. `frontend/web.py` gestisce redirect e messaggi di errore dei form; le rotte web sono sotto `/web` e non compaiono nello schema OpenAPI.

Le rotte JSON principali sono:

Prefisso	Azioni
`/auth`	Registrazione, login, utente corrente, logout.

Prefisso	Azioni
`/exams`	Lista, creazione, eliminazione, caricamento/eliminazione PDF.
`/exams/{exam_id}`	Rinomina, elenco/verifica/reindex materiali, appunti, dubbi e cronologia chat.
`/sessions`	Creazione e lista sessioni.
`/sessions/{session_id}`	Quiz, tutor, scelta domanda, pausa/ripresa, chiusura, report e appunti.
`/health`	Controllo HTTP basilare.

Le forme complete di payload, status e route sono definite negli schemi Pydantic e in `/docs`. La maggior parte delle rotte API richiede `student_id` come query parameter, coerente con il proprietario autenticato; le risorse altrui vengono nascoste con risposta 404. La conoscenza di un ID non autorizza quindi la lettura della risorsa.

10. Autenticazione e sicurezza

L'autenticazione risponde alla domanda «chi sta effettuando la richiesta?», mentre l'autorizzazione stabilisce «quali risorse può utilizzare?». In StudyCoach il secondo controllo è essenziale perché più studenti possono avere esami e documenti con nomi simili, ma con proprietari diversi. Il possesso di un identificatore di esame o di sessione non costituisce un permesso di accesso.

La password viene usata nella verifica iniziale delle credenziali; le richieste successive impiegano un token di sessione. Conservare il digest del token nel database consente al server di confrontare la credenziale ricevuta senza memorizzarne direttamente il valore utilizzabile. Scadenza e revoca tramite logout definiscono il periodo nel quale la credenziale rimane valida.

Le password sono verificate con bcrypt. Login e registrazione rilasciano un token casuale; il browser lo conserva nel cookie `sc_token`, mentre il server salva un digest e una scadenza di sette giorni. Le API accettano cookie o `Authorization: Bearer <token>`. Il logout invalida il token lato server. `authorization.py` controlla studente, esame e sessione prima di eseguire gli handler. Il login ha un limite di tentativi per account e origine, registrato in SQLite.

Il middleware controlla l'header `Origin` per i POST sotto `/web/` e respinge origini diverse dall'host della richiesta. La configurazione CORS non abilita automaticamente origini esterne: la lista predefinita è vuota. `security.py` normalizza i nomi di file; i messaggi pubblici degli errori passano per funzioni che evitano di esporre dettagli tecnici o chiavi API. L'esposizione in rete pubblica richiede inoltre TLS, gestione dei backup, protezione dei segreti e procedure operative adeguate.

11. Osservabilità e gestione errori

L'osservabilità permette di collegare un comportamento visibile allo studente alle operazioni eseguite dal server. Una richiesta può attraversare autenticazione, ricerca nei documenti, chiamata al provider e salvataggio: conoscere soltanto l'errore finale non basta a individuare la fase che lo ha causato. L'identificatore della richiesta fornisce un riferimento comune per ricostruire questi passaggi.

La gestione degli errori distingue la diagnostica interna dal messaggio pubblico. Lo studente deve ricevere un'indicazione comprensibile e utile al recupero; il log deve consentire l'analisi tecnica senza esporre inutilmente credenziali o contenuti personali. Anche un controllo di disponibilità va interpretato in base al suo ambito: un

processo che risponde via HTTP può avere un indice documentale incoerente o un provider temporaneamente indisponibile.

Ogni risposta gestita dal middleware HTTP include `X-Request-ID`. `backend/observability.py` usa l'ID nel contesto della richiesta e registra stadi come HTTP, retrieval e chiamate al modello con tempi ed esiti. I log non dovrebbero contenere prompt integrali, risposte degli studenti o brani dei PDF. Il codice distingue limiti quota/rate limit, errori temporanei del provider, PDF non leggibili, indice mancante e materiali incoerenti; la UI mostra messaggi orientati al recupero. `/health` prova soltanto che il processo risponde: non verifica DB, Chroma o provider esterni.

12. Test e valutazione

La verifica del progetto richiede di separare il comportamento del software dalla qualità dei contenuti prodotti. Un test automatico può accertare che una risorsa altrui venga rifiutata, che una risposta sia salvata o che una selezione di fonti diventi un filtro di ricerca. Questi sono vincoli applicativi riproducibili. Stabilire se una domanda sia pertinente o una valutazione sia convincente richiede invece anche prove sul modello e un riferimento disciplinare.

L'uso di sostituti deterministici rende i test indipendenti da latenza, quote e variabilità dei servizi remoti. È una scelta utile per controllare le regole del programma, purché i risultati non vengano interpretati come misure della qualità linguistica. Le esecuzioni con provider reali aggiungono un diverso tipo di evidenza e devono conservare le condizioni nelle quali sono state svolte.

Pytest esclude per impostazione predefinita i test marcati `real_llm`; questi richiedono credenziali e possono consumare quota. I test in `test/` coprono autenticazione, proprietà delle risorse, API/UI, ingestione, sorgenti, quiz, tutor, valutazione, sessioni, tempo, report, note, configurazione e osservabilità. Test con embedding o provider simulati verificano logica e isolamento, non la qualità della ricerca semantica o del modello in produzione.

Il runner `eval/run_eval.py` controlla schema e hash dei casi di validazione. L'esecuzione con Groq richiede `GROQ_API_KEY`, salva risultati JSON/Markdown in `eval/results/` e misura, tra l'altro, voto e citazioni. Il runner invia il testo di riferimento completo a Groq e **non attraversa il retrieval dell'applicazione**: i suoi risultati non dimostrano da soli l'affidabilità end to end del flusso PDF → Chroma → tutor. I risultati già presenti in `eval/results/` descrivono le esecuzioni in cui sono stati prodotti.

13. Procedure operative e recupero

Le procedure operative permettono di mantenere utilizzabile l'applicazione quando una dipendenza non risponde o quando lo stato dei dati risulta incompleto. Il primo passo consiste nell'individuare il componente coinvolto: ambiente di esecuzione, documento sorgente, indice vettoriale, provider oppure file derivato. Una stessa azione visibile, come la generazione di un quiz, può dipendere da tutti questi elementi.

Il recupero deve partire dal dato autorevole per l'operazione. Per ricostruire l'indice occorre il PDF conservato; per rigenerare un report occorre il log della sessione. La verifica dei materiali permette di controllare se archivio documentale e indice descrivono ancora lo stesso insieme di fonti. Il backup deve quindi mantenere coerenti gli archivi che partecipano al flusso.

Sintomo	Controllo e azione
Il server non parte	Verificare versione Python, dipendenze, traceback completo e presenza di `config/app.toml`.
Il primo caricamento PDF sembra lento	Attendere download/cache del modello di embedding e OCR delle pagine senza testo.
Materiale incoerente	Usare la verifica dell'esame, controllare file in `data/uploads`, hash e conteggi; reindicizzare dal PDF originale.
Sono cambiati i parametri di chunking	Reindicizzare le fonti; i vecchi chunk mantengono i vecchi confini finché non sono ricreati.
Quota o timeout LLM	Verificare provider, modello, chiave e tempi comunicati dall'errore; riprovare dopo l'intervallo indicato.
Report mancante	Se la sessione ha voto e log valutato, usare la rigenerazione del report.
Ripristino da backup	Ripristinare insieme DB, PDF, indice Chroma e report; poi verificare materiali ed eseguire i flussi essenziali.

La cancellazione di fonti o esami coinvolge più store (SQLite, file e Chroma). I moduli `materials.py` ed `exam_deletion.py` implementano pulizia e recupero in caso di errori; non esiste però una transazione distribuita atomica fra questi sistemi. Dopo un'interruzione del processo durante una modifica, usare la verifica dei materiali prima di generare nuovo contenuto.

14. Limiti tecnici e punti di estensione

I confini tecnici definiscono le condizioni nelle quali interpretare le funzionalità descritte. Il sistema applica controlli su struttura, citazioni e coerenza dei dati, ma la generazione conserva una componente probabilistica. La qualità del testo estratto dal PDF influenza il recupero, e il materiale recuperato influenza a sua volta ciò che il modello può giustificare.

Anche l'organizzazione dell'esecuzione ha conseguenze operative. Dati locali e richieste che svolgono direttamente elaborazioni lunghe costituiscono un assetto concreto, da considerare nel dimensionamento e nella gestione delle interruzioni. Le osservazioni seguenti descrivono tali condizioni e i punti dell'architettura coinvolti in un'eventuale evoluzione.

La correttezza disciplinare delle domande, dei giudizi e degli appunti dipende dalla qualità del PDF, dall'estrazione, dal retrieval e dal modello remoto. Una citazione letterale prova che il brano esiste nel testo indicizzato; non prova automaticamente che la domanda sia didatticamente valida. SQLite e Chroma sono locali: l'assetto corrente è pensato per una singola installazione, non per più processi con storage indipendente. Le operazioni PDF e LLM sono eseguite durante richieste HTTP; per carichi elevati occorre una coda di job persistenti. La migrazione delle tabelle usa principalmente funzioni `_ensure_*` in `backend/db/models.py`: esiste la configurazione Alembic, ma non una serie di revisioni versionate.

15. Organizzazione della manutenzione

La manutenzione richiede di seguire il percorso del dato attraverso i livelli. Una modifica al form può richiedere un adeguamento dello schema; una nuova informazione persistita può richiedere una lettura compatibile con le sessioni precedenti; una variazione nell'indicizzazione può rendere necessario ricostruire i vettori. Limitarsi al punto in cui un valore viene visualizzato non è sufficiente per comprenderne l'effetto sull'intero flusso.

La suddivisione in moduli permette di individuare i componenti responsabili. Il router definisce ingresso e uscita, il servizio realizza la regola applicativa e il modello dati conserva il risultato. La verifica di una modifica deve quindi includere il contratto di accesso, la logica e gli effetti persistenti, soprattutto quando intervengono sia SQLite sia archivi esterni al database.

Le modifiche ai flussi partono dal router in `backend/api/` o `frontend/web.py` e proseguono nel servizio in `backend/agents/`, negli schemi di `backend/models/schemas.py` e nella persistenza di `backend/db/models.py`. Un cambiamento del formato JSON salvato nelle sessioni deve mantenere la lettura dei dati storici. Un cambiamento a chunking o modello di embedding richiede reindex e verifica Chroma. I test devono coprire casi positivi, negativi, proprietà delle risorse e fallimenti di persistenza, oltre alla regressione completa.

16. Analisi dettagliata dei livelli

L'analisi per livelli approfondisce il modo in cui l'applicazione passa dall'avvio alla gestione delle richieste. Le sezioni precedenti hanno presentato i componenti e le funzioni; qui vengono esaminati l'ordine di inizializzazione, il ruolo degli adattatori web e la distribuzione delle responsabilità fra i moduli. Questa lettura permette di collegare i nomi dei file alle operazioni effettivamente svolte.

Il livello applicativo non coincide con il solo modello linguistico. Preparazione del contesto, controllo delle credenziali, validazione, scelta del provider e persistenza sono attività del programma. Gli agenti partecipano a questo percorso con compiti specifici e producono risultati che il codice deve ancora controllare prima di renderli disponibili.

16.1 Processo e ordine di inizializzazione

L'import di `backend` esegue `backend/__init__.py`: legge `.env` senza sovrascrivere l'ambiente di processo, apre `config/app.toml`, controlla che i nomi siano maiuscoli e i valori siano scalari, quindi applica i default con `os.environ.setdefault`. Alcune costanti nei moduli leggono l'ambiente al momento dell'import: dopo un cambio di configurazione è quindi opportuno riavviare il processo. `backend/api/main.py` costruisce l'app FastAPI, monta i router e gli asset e nel lifespan richiama `init_db()` prima di dichiarare il server pronto.

Il middleware di tracing crea un ID per richiesta, misura la durata e aggiunge `X-Request-ID` alle risposte gestite. Il middleware dei form controlla `Origin` se presente sui POST `/web/`. FastAPI esegue molte funzioni sincrone in threadpool; il router web usa anche `run_in_threadpool` e `asyncio.to_thread` per parsing, retrieval e generazione. Questo evita di occupare il loop asincrono, ma non fornisce una coda di lavoro durevole. `/health` verifica soltanto che il processo HTTP risponde; non apre Chroma né chiama provider esterni.

16.2 Confine tra browser, API e servizi

`frontend/web.py` è l'adattatore browser. Riceve form, identifica lo studente dal cookie, verifica l'esame o la sessione, richiama funzioni di `backend/api/` e rende una pagina Jinja oppure restituisce un redirect. Le rotte web sono escluse dallo schema OpenAPI. Gli endpoint JSON usano Pydantic per validare payload e risposte. Le pagine HTML non sono la fonte primaria dello stato: dopo refresh o redirect rileggono SQLite.

I template sono `base.html`, `auth.html`, `exams.html`, `exam.html`, `setup.html`, `study.html`, `chat.html`, `chat_history.html` e `history.html`. Gli asset JavaScript gestiscono timer, invio chat e avanzamento delle operazioni. Quando il browser invia `Accept: application/x-ndjson`, `_operation_stream()` produce eventi `progress`, `complete` ed `error`, più heartbeat circa ogni 15 secondi. `complete` contiene un URL di destinazione. `_notes_stream()` invia anche il PDF finale in base64 e il Markdown degli appunti; senza tale header, il form segue il percorso ordinario. Lo stream è legato alla connessione HTTP e non equivale a un job recuperabile dopo chiusura del browser.

16.3 Moduli di dominio

`ingestor.py` possiede lettura PDF, OCR, chunk, embedding, collection Chroma, ricerca e verifica delle fonti. `quiz_generator.py` legge passaggi indicizzati e produce domande fondate. `socratic_tutor.py` mantiene la macchina a stati del turno e il tutor dei dubbi. `evaluation_validator.py` e `quality_controls.py` applicano regole deterministiche alle risposte del modello. `adaptive_engine.py` sceglie le domande e calcola statistiche. `notes_generator.py` costruisce appunti e PDF; `report_generator.py` costruisce report Markdown/PDF. `model_router.py` sceglie provider e modello per funzionalità; `llm_errors.py` classifica errori riprovabili e limiti quota.

I prompt sono file versionati in `backend/agents/prompts/`. Cambiare il contenuto di un prompt mantenendo la stessa versione rende meno interpretabili i risultati storici. La versione del prompt del quiz è salvata nel risultato. `materials.py` ed `exam_deletion.py` coordinano modifiche fra più store; `security.py`, `session_cookie.py` e `login_limiter.py` gestiscono credenziali, cookie e tentativi di accesso.

16.4 Dipendenze e ruolo

Tecnologia	Uso concreto
FastAPI, Uvicorn, Starlette	Server ASGI, routing, middleware, streaming e file statici.
Jinja2, MarkdownIt, MarkupSafe	Pagine HTML e rendering degli appunti nella cronologia.
SQLAlchemy, SQLite	Dati relazionali e transazioni locali.
ChromaDB, sentence-transformers	Indice vettoriale persistente ed embedding locale.
pypdf, pypdfium2, RapidOCR, NumPy	Estrazione PDF, rendering pagine e OCR.
Agno e SDK provider	Chiamate a Groq, Mistral e Gemini.
fpdf2 e font inclusi	Creazione di report e appunti PDF.
bcrypt	Hash e verifica password.
pytest e httpx	Test automatici dell'extra `dev`.

Le dipendenze sono dichiarate in `pyproject.toml`; `uv.lock` registra le versioni risolte per rendere riproducibile l'ambiente.

17. Contratti HTTP e flussi completi

Un contratto HTTP comprende il metodo, il percorso, i dati accettati, i controlli richiesti e l'effetto dell'operazione. Per comprendere il comportamento dell'applicazione occorre leggere tali contratti insieme alla sequenza che li collega. Creare una sessione, generare il quiz e avviare un turno sono passaggi distinti, anche quando la pagina web li presenta come un'unica azione.

I flussi seguenti mostrano dove il sistema valida i dati, dove effettua chiamate esterne e in quale momento registra i risultati. Particolare attenzione è rivolta ai fallimenti intermedi: la gestione di una richiesta non termina quando il modello restituisce del testo, ma quando l'applicazione ha verificato e conservato ciò che serve al passo successivo.

17.1 Catalogo delle API JSON

La tabella seguente collega le operazioni esposte ai moduli responsabili. Per parametri e schemi completi consultare `/docs` e `backend/models/schemas.py`. Le rotte di esami e sessioni richiedono autenticazione e controllo del proprietario; quando previsto, `student_id` in query deve coincidere con il token.

Metodo e percorso	Effetto principale	Modulo
`POST /auth/register`, `POST /auth/login`	Crea account o autentica e imposta cookie.	`api/auth.py`
`GET /auth/me`, `POST /auth/logout`	Identità corrente e revoca credenziale.	`api/auth.py`
`GET /exams`, `POST /exams`, `DELETE /exams/{exam_id}`	Lista, creazione, cancellazione.	`api/exams.py`
`POST /exams/{exam_id}/ingest`	Upload e indicizzazione multipart.	`api/exams.py`
`DELETE /exams/{exam_id}/materials/{material_id}`	Cancellazione fonte.	`api/exams.py`
`GET /exams/{exam_id}/materials`, `PATCH /exams/{exam_id}`	Lista fonti e rinomina esame.	`api/frontend_extras.py`
`POST /exams/{exam_id}/verify`, `/reindex`	Verifica integrità e ricostruzione indice.	`api/frontend_extras.py`
`POST /exams/{exam_id}/notes`, `/notes/pdf`	Appunti testuali o PDF.	`api/frontend_extras.py`
`POST /exams/{exam_id}/doubts`	Apre dubbio e domanda guida.	`api/frontend_extras.py`
`POST /exams/{exam_id}/doubts/{conversation_id}/answer`	Valuta risposta al dubbio.	`api/frontend_extras.py`
`DELETE /exams/{exam_id}/doubts/{conversation_id}`	Scarta dubbio.	`api/frontend_extras.py`
`POST /sessions`, `GET /sessions`	Crea sessione o elenca quelle di un esame.	`api/sessions.py`
`POST /sessions/{session_id}/quiz`	Genera e salva quiz.	`api/sessions.py`
`POST /sessions/{session_id}/tutor/start`	Avvia turno con una domanda canonica.	`api/sessions.py`

Metodo e percorso	Effetto principale	Modulo
`POST /sessions/{session_id}/tutor/answer1`, `/answer2`	Domanda guida e valutazione.	`api/sessions.py`
`POST /sessions/{session_id}/tutor/contest`, `/close`	Contestazione e chiusura turno.	`api/sessions.py`
`POST /sessions/{session_id}/next-question`, `/close`	Scelta adattiva e report.	`api/sessions.py`
`GET /sessions/{session_id}`, `/resume`, `/detail`	Stato sintetico e ricostruito.	`api/sessions.py`, `api/session_extras.py`
`POST /sessions/{session_id}/interrupt`, `/resume-active`, `/mark-complete`	Lifecycle di una sessione incompleta.	`api/session_extras.py`
`POST /sessions/{session_id}/timer`	Tempo attivo.	`api/session_extras.py`
`DELETE /sessions/{session_id}`	Elimina sessione non completata.	`api/session_extras.py`
`POST /sessions/{session_id}/notes`	Appunti dal log.	`api/session_extras.py`
`GET /sessions/{session_id}/report`, `POST /sessions/{session_id}/report/regenerate`	Download e rigenerazione.	`api/session_extras.py`

17.2 Registrazione, login e accesso alle risorse

`RegisterRequest` impone username di 3–50 caratteri, email formalmente valida, password di almeno 6 caratteri e conferma uguale; la password è limitata a 72 byte UTF-8 per bcrypt. Il server salva hash password, non la password. `auth.login()` verifica le credenziali e consulta il limite dei tentativi di accesso registrato in `login_attempts`. Il token casuale è consegnato tramite cookie; nel DB viene salvato il suo digest e una scadenza di sette giorni. Le API accettano il cookie oppure `Authorization: Bearer <token>`. `authorization.py` ricava il proprietario, verifica `student_id` e le relazioni esame/sessione prima dell'handler. L'accesso a una risorsa altrui restituisce 404 per non confermarne l'esistenza.

17.3 Upload di un PDF: sequenza e compensazioni

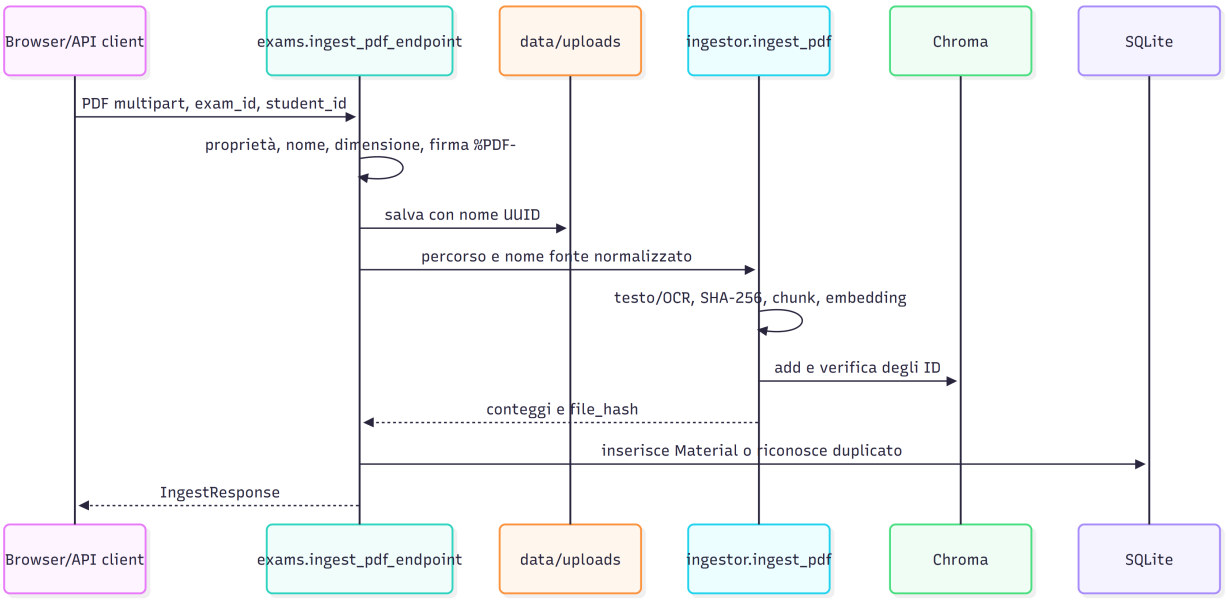


Figura 3. Sequenza di caricamento e indicizzazione di un PDF.

L'endpoint legge al massimo `MAX_UPLOAD_BYTES + 1` byte, così non alloca un upload illimitato. Controlla estensione `.pdf` e firma iniziale `%PDF-`; un file oltre limite produce 413. Il file persistito ha nome UUID e il nome originale, normalizzato da `safe_name()`, resta nei metadati. L'ingestor calcola SHA-256 dei byte, usa `pypdf` per estrazione testo pagina per pagina e RapidOCR sulle sole pagine senza testo. Tiene distinti pagine totali, con testo, scartate e OCR. Rifiuta PDF protetti/corrotti, senza testo utilizzabile o senza chunk validi.

I chunk vengono inseriti in batch da 100 e gli ID effettivamente presenti vengono confrontati con quelli attesi. Se l'inserimento fallisce, il codice prova a cancellare i chunk parziali. Se il DB non riesce a salvare `Material`, la routine di rollback dell'ingestione cancella i chunk nuovi **solo se** nessun materiale già registrato possiede lo stesso hash. Se invece un PDF identico era già presente nell'esame, il nuovo file viene eliminato e il record esistente resta. Questi passaggi sono compensazioni fra sistemi separati, non una transazione atomica comune.

17.4 Dalla selezione delle fonti al quiz

Il browser invia gli ID delle fonti dell'esame. `_selected_materials()` verifica che tutti appartengano allo stesso esame e `verify_exam_material()` confronta file, hash e chunk. La UI ammette da 3 a 10 domande, mentre lo schema JSON di `QuizGenerateRequest` ammette da 1 a 20: il limite web è più stretto. `_start_study_work()` crea una sessione, genera il quiz e avvia la prima domanda. Se questa sequenza web fallisce dopo la creazione, cancella la sessione incompleta. Un client che usa direttamente le API gestisce invece creazione e quiz come operazioni separate.

`create_session()` salva stato `in_corso`, inizio e tempo attivo. `generate_session_quiz()` riverifica le fonti, genera esattamente il numero richiesto di domande e salva in `quiz_json` il risultato e gli eventuali `material_ids`. `tutor_start()` accetta solo una domanda uguale a quella canonica persistita e rifiuta domande già chiuse o un secondo turno aperto.

17.5 Macchina a stati di una domanda

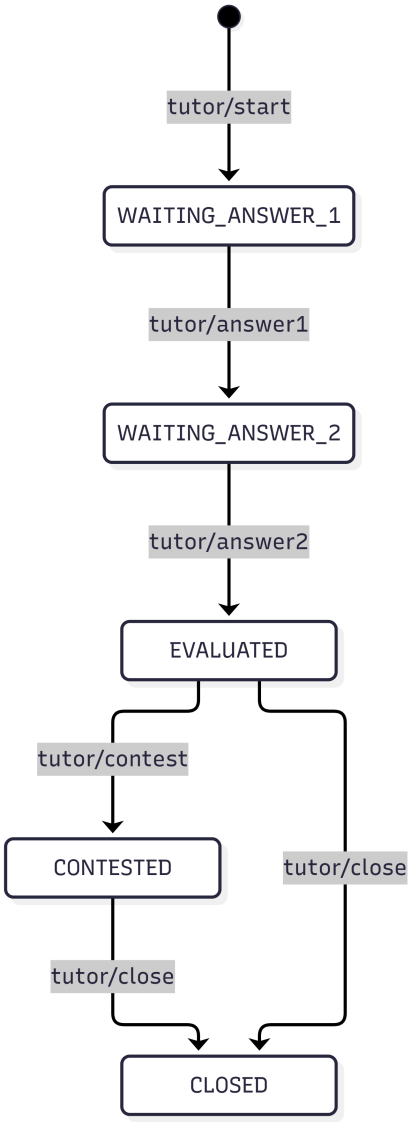


Figura 4. Macchina a stati del turno di una domanda.

`answer1` salva il testo della prima risposta **prima** di chiamare il modello per la domanda guida. `answer2` salva la seconda risposta prima della valutazione. Se la chiamata LLM fallisce, il testo inviato rimane nel turno persistito. A successo vengono salvati domanda guida, feedback, voto e motivazione. Dopo ogni richiesta il tutor viene ricostruito da `current_turn_json` e recupera di nuovo il contesto Chroma delle fonti selezionate. Gli stati sono definiti nell'enum `TurnState` di `socratic_tutor.py`.

La valutazione può richiedere due giudizi indipendenti. `_evaluate_one()` valida JSON, coerenza fra voto e punti corretti/mancanti, sufficienza del materiale e contenuto effettivo della risposta. Per un output valido il punteggio viene calcolato tramite `rubric_score()`, poi corretto dai guardrail. Fra due voti affidabili si sceglie il minore; una differenza superiore a un punto marca il risultato `reliable=False`. Un output non validabile dopo i tentativi può essere restituito con voto numerico ma marcato non affidabile; in assenza di materiale sufficiente l'operazione fallisce. La contestazione è opzionale e salva giudizio sulla contestazione, eventuale differenza di voto e risposta allo studente.

`tutor/close` richiede un voto finale. Serializza il tentativo in `qa_log` e cancella `current_turn_json` nella **stessa transazione SQLite**, dopo aver verificato che l'ID della domanda non sia già presente. Questo è il punto in cui un

turno diventa una risposta chiusa e conteggiabile.

17.6 Sessione, tempo, adattamento e report

`next-question` ricava le domande rimanenti dal quiz e dal `qa_log` salvati, anziché fidarsi della lista inviata dal client. Ricostruisce `AdaptiveEngine` escludendo i tentativi inaffidabili, poi sceglie nell'ordine un argomento debole, un argomento non ancora trattato oppure la prima domanda rimasta. `close_session()` ricostruisce l'engine includendo i voti numerici anche se marcati non affidabili, richiede almeno una risposta valutata e crea report Markdown/PDF. Aggiorna voto finale, percorso PDF, fine, durata e stato `completata` in SQLite. Se il salvataggio nel DB fallisce, tenta di ripristinare la precedente versione dei file del report.

`interrupt` congela il tempo e passa a `interrotta` conservando quiz, turno e log. `resume-active` riporta una sessione riprendibile a `in_corso` e riavvia il cronometro. `mark-complete` chiude un'incompleta senza voto finale, conservando lo stato `interrotta`. La cancellazione è consentita soltanto se non è completata e `score_total` è nullo. `GET /sessions/{id}/resume` restituisce quiz, turno parziale e statistiche adattive ricostruite.

17.7 Chat dei dubbi e appunti

La chat dei dubbi non risponde subito con una soluzione libera. `start_doubt()` cerca passaggi pertinenti, chiede al `DoubtTutor` una domanda guida e salva una `DoubtConversation` in un thread. Un dubbio già aperto nello stesso thread deve essere risolto o scartato; se la richiesta è identica, viene riusato il record aperto. `answer_doubt()` ricostruisce il tutor, ricerca di nuovo i passaggi e salva risposta e valutazione nel record. La UI permette più thread, cronologia ed eliminazione di un thread.

17.7.1 Origine e suddivisione del materiale

`generate_notes_pdf_from_materials()` riceve i materiali selezionati per l'esame. Per ciascuno interroga i passaggi indicizzati usando l'hash del file, oppure il nome del file conservato per i dati precedenti che non hanno l'hash. Se una fonte non ha contenuto indicizzato, la generazione si interrompe con un errore che indica quella fonte: non viene sostituita silenziosamente con un altro PDF. `_passage_contexts()` conserva le etichette di fonte e pagina e suddivide il testo in gruppi entro `NOTES_MAX_CONTEXT_CHARS`; un passaggio lungo viene spezzato rispettando `NOTES_MAX_CHUNK_CHARS`. I gruppi sono elaborati nell'ordine dei passaggi recuperati. La procedura produce una sezione per ciascuna fonte e, quando una fonte richiede più gruppi, una parte per ciascun gruppo.

17.7.2 Team, controllo della delega e revisione

Per ogni gruppo `_notes_from_passages()` prepara il prompt con il materiale e avvia `NotesTeam` in modalità Agno `coordinate`. Il Team ha un leader e il membro `NotesGeneratorAgent`: il leader affida al membro la stesura e restituisce gli appunti in Markdown. `_check_notes_team_members()` controlla l'esecuzione effettiva del membro prima di accettare la risposta; il solo testo finale del leader non prova che sia avvenuta la delega. Questa è la parte multi-agent. L'ordine dei gruppi, la validazione e la composizione finale sono gestiti dal codice applicativo: formano una pipeline attorno al Team.

`_run_with_retry()` controlla la struttura della risposta, in particolare il titolo di capitolo e il `## Riepilogo` finale. Per un riepilogo assente o vuoto può ricostruire la sezione usando una riga già presente nel corpo, senza introdurre nuove affermazioni. Una risposta non valida può causare una nuova richiesta entro il limite `NOTES_MAX_LLM_ATTEMPTS`. La riga di log `validazione=ok` indica che **questa risposta del Team** ha superato il controllo strutturale; non indica che la revisione, gli altri gruppi o il PDF siano già terminati.

Solo dopo il Team, `_verify_notes_fidelity()` chiama `NotesFidelityReviewAgent` con il testo prodotto e il gruppo di materiale originale. Il revisore restituisce un oggetto JSON contenente i frammenti che considera non supportati; il codice richiede che ogni frammento sia copiato esattamente dagli appunti e rimuove le righe corrispondenti. Se

viene segnalato un titolo, conserva il titolo strutturale del riepilogo oppure lo sostituisce con un titolo neutro. Ripristina, se necessario, un riepilogo a partire dal testo rimasto e ricontrolla la struttura. Una risposta JSON invalida viene ritentata entro `NOTES_MAX_REVIEW_ATTEMPTS`; se la verifica non riesce, la generazione fallisce invece di presentare il testo come verificato. Il revisore è un agente distinto, ma viene eseguito **dopo** il Team, non come suo membro.

17.7.3 Provider, quote e stato dell'operazione

La configurazione predefinita sceglie Gemini `gemini-3.8-flash` per gli appunti e Groq `qwen/qwen3.8-27b` come riserva. Il secondo modello usa la chiave `GROQ_API_KEY`; non esiste una chiave Qwen distinta nell'applicazione. In presenza di un errore temporaneo del servizio principale, di un limite quota o di una delega incompleta, la scrittura può passare al provider di riserva. La scelta viene mantenuta per i gruppi successivi della stessa fonte. La revisione usa il provider selezionato per il gruppo e, quando parte dal provider principale, può a sua volta passare alla riserva. I limiti di token del fallback sono configurati separatamente da quelli del provider principale. Il passaggio di provider non garantisce disponibilità: ciascun servizio applica le proprie quote.

Il callback di avanzamento segnala lettura, generazione, revisione, completamento della parte, completamento della fonte e costruzione del PDF. La UI riceve questi eventi mentre la richiesta è aperta. Il PDF viene reso disponibile soltanto dopo che tutte le parti e tutte le fonti hanno superato le rispettive fasi. Un errore successivo a `validazione=ok`, per esempio durante la revisione o la generazione di un'altra parte, può quindi impedire il download.

17.7.4 Composizione del testo e impaginazione PDF

`_merge_note_sections()` unisce le parti in ordine e raccoglie i punti di riepilogo in una sola sezione finale, evitando i duplicati testuali. `generate_notes_pdf_from_materials()` crea un frontespizio e una sezione separata per ciascuna fonte; opzionalmente restituisce anche il Markdown insieme ai byte del PDF, usati dall'interfaccia per il download. Il PDF degli appunti viene prodotto nella richiesta e trasmesso al browser: non è registrato come report di sessione in `sessions.report_path`.

`_render_markdown()` interpreta titoli, separatori, citazioni, elenchi e paragrafi. Una tabella viene riconosciuta se una riga di intestazione è seguita da un separatore Markdown valido, per esempio `| --- | --- |`, e da righe con lo stesso numero di colonne. `_table_cells()` divide le celle rispettando i caratteri `|` preceduti da escape. `_render_table()` calcola le larghezze delle colonne in base al contenuto entro la larghezza utile della pagina, misura le righe a capo prima di disegnarle e crea celle con bordi. Quando lo spazio non basta sposta una riga intera alla pagina successiva, se possibile; per una riga eccezionalmente alta la divide tra pagine. In entrambi i casi ripete l'intestazione. Il renderer tratta una tabella malformata come testo ordinario e rimuove la marcatura inline Markdown nelle celle; non gestisce fusioni di celle, tabelle annidate o una riproduzione tipografica completa del PDF sorgente. Una tabella presente come immagine nel materiale passa prima dall'OCR: la qualità della struttura generata dipende dal testo estratto e dagli appunti prodotti dal modello.

`generate_notes_from_log()` segue un percorso diverso: usa le risposte della sessione, individua argomenti deboli e, se richiesto, aggiunge passaggi RAG filtrati dai `material_ids` persistiti nel quiz. Questo percorso usa un singolo agente di scrittura e non `NotesTeam`. I controlli descritti riducono errori evidenti, ma non certificano l'accuratezza disciplinare, soprattutto per formule, OCR imperfetto e tabelle complesse.

18. Algoritmi e dati persistiti

Gli algoritmi descritti in questo capitolo trasformano informazioni documentali e risultati di studio in scelte applicative. Il chunking determina le unità disponibili al recupero; il filtro delle fonti delimita il materiale consentito; la selezione dei contesti distribuisce la copertura del quiz. Il calcolo del tempo e l'adattamento utilizzano invece lo stato delle sessioni e i tentativi chiusi.

La conservazione degli identificatori è il collegamento fra questi processi. L'hash riconosce il contenuto di un PDF, l'ID del materiale lo collega all'esame e l'ID della domanda permette di distinguere i turni del quiz. Se questi riferimenti vengono preservati, il sistema può controllare la provenienza e ricostruire il percorso senza affidarsi al solo testo visualizzato.

18.1 Relazioni e campi di stato

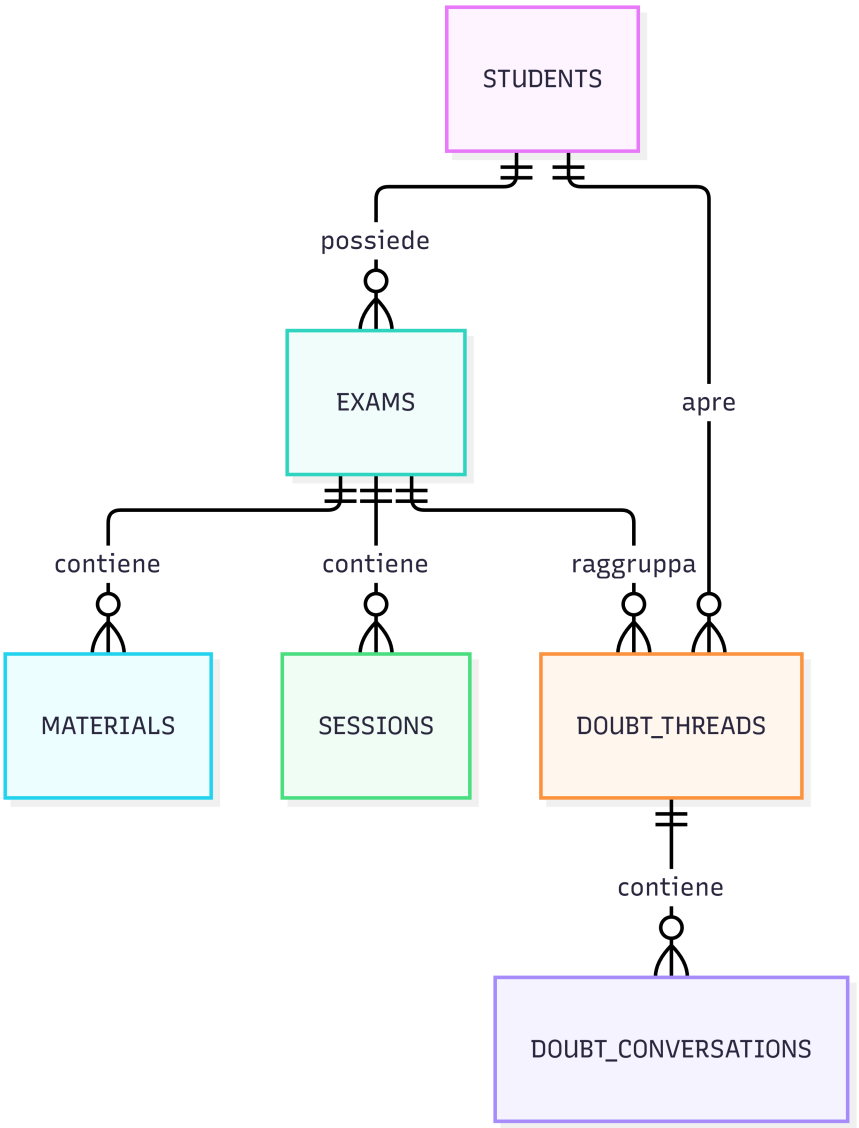


Figura 5. Relazioni principali fra le entità persistite.

Le chiavi primarie di `students`, `exams`, `materials` e `sessions` sono interi; thread e conversazioni usano UUID testuali. `sessions.student_id` può essere nullo nei dati legacy, mentre la proprietà resta ricostruibile dall'esame. `quiz_json`, `current_turn_json` e `qa_log` sono colonne `TEXT` contenenti JSON, non tabelle normalizzate. L'evoluzione del

formato richiede una lettura compatibile con le sessioni precedenti. Le date nuove sono UTC; `session_clock.py` tratta timestamp legacy senza timezone come UTC nel calcolo della durata.

Campo	Scrittura e significato	Lettori
`materials.file_hash`	SHA-256 dei byte PDF in upload o reindex.	Deduplica, verifica, filtri Chroma e cancellazione.
`materials.chunks_count`	Conteggio dei chunk indicizzati.	Verifica di consistenza.
`materials.pages_with_text`, `pages_skipped`	Esito estrazione/OCR per pagina.	Interfaccia esame.
`sessions.quiz_json`	Domande, versione prompt e `material_ids`.	Tutor, ripresa, scelta domanda e note.
`sessions.current_turn_json`	Domanda attiva, risposte, guida, voto e contestazione.	Ripresa del tutor e UI.
`sessions.qa_log`	Lista di turni chiusi e voti finali.	Adattamento, report, appunti, cronologia.
`sessions.duration_seconds`	Secondi attivi già accumulati.	Timer e cronologia.
`sessions.active_started_at`	Inizio del segmento attivo.	Calcolo durata fino a pausa/chiusura.
`sessions.score_total`	Media finale della sessione.	Elenco esami e cronologia.
`sessions.report_path`	Percorso del PDF finale.	Download e rigenerazione.

18.2 Chunking e identità della fonte

`chunk_text()` divide per parole, preferisce fermarsi a fine frase o paragrafo entro `RAG_CHUNK_WORDS` (default 90) e usa `RAG_CHUNK_OVERLAP_WORDS` (default 15) per sovrapporre il contesto. `_model_safe_chunks()` suddivide ulteriormente un chunk quando supererebbe `max_seq_length` del tokenizer dell'embedder. I pezzi sotto 20 caratteri non vengono indicizzati. Un ID Chroma ha forma `<sha256>_p<pagina>_c<indice>`: l'hash differenzia PDF, la pagina mantiene la provenienza e l'indice identifica il chunk. Gli ID attesi sono ricalcolabili se PDF, modello e parametri di chunking restano invariati.

La collection dell'esame contiene metadati `student_id`, `exam_id`, `embedding_model`; ogni documento contiene pagina, fonte e hash. `get_collection()` confronta il modello di embedding registrato con quello corrente. Se cambia, ricalcola gli embedding dei documenti presenti in batch e poi aggiorna il metadato della collection. Questa conversione riguarda i vettori, non ricrea i confini dei chunk. Il cambio di modello richiede un backup preventivo di `data/chroma` e una verifica dell'indice dopo la conversione.

18.3 Ricerca e filtro delle fonti

`retrieve_passages()` legge passaggi indicizzati e li ordina per fonte, pagina e indice. È usato quando serve coprire un documento intero, ma può costare per PDF molto lunghi. `search_passages()` usa la query vettoriale e restituisce testo, fonte, pagina, ID e distanza. `retrieve_chunks()` espone una variante solo testuale. Un `material_ids=[]` produce una lista vuota. Con ID presenti, `_material_where_clause()` verifica l'appartenenza all'esame e costruisce un filtro Chroma basato su `file_hash`. Per record legacy senza hash usa il nome della fonte; fonti omonime in quel caso richiedono reindex per togliere l'ambiguità. `RETRIEVAL_MAX_DISTANCE`, se impostato, filtra i risultati troppo distanti.

18.4 Copertura e controllo del quiz

`_quiz_contexts()` attraversa le fonti a rotazione, etichetta ogni passaggio con nome e pagina e costruisce gruppi entro `QUIZ_MAX_CONTEXT_CHARS`. `_select_quiz_contexts()` limita le chiamate mantenendo almeno un gruppo per fonte e scegliendo anche inizio/fine e posizioni distribuite nel documento. Se le fonti distinte superano il normale budget `QUIZ_MAX_BATCHES`, il budget può aumentare per coprirle tutte almeno una volta.

Il modello produce JSON validato da `QuizPayload`. Ogni `source_quote` viene normalizzato per spazi e maiuscole e cercato **dentro un singolo passaggio** del gruppo; una frase inventata o fusa da più brani viene respinta. Un gruppo invalido può ricevere un tentativo correttivo. Il testo delle domande è deduplicato. Se il provider primario raggiunge la quota e c'è un fallback configurato, il generatore può continuare sul provider di riserva. Il recupero delle domande mancanti usa solo contesti che hanno già dato risultati verificabili. Infine l'endpoint rifiuta un quiz che contenga meno domande di quelle richieste, anche se quelle prodotte sono valide.

18.5 Calcolo del tempo e dei voti

Il tempo attivo visualizzato è `duration_seconds` accumulato più i secondi trascorsi da `active_started_at` **soltanto se** lo stato è `in_corso`. `pause_clock()` somma il segmento e azzera l'inizio attivo; `resume_clock()` apre un segmento soltanto se non è già aperto. L'adattamento esclude tentativi con `reliable=False` a livello di record o valutazione. Il report storico finale include i voti numerici disponibili anche se marcati non affidabili. Un argomento è debole con media sotto 6, oppure con un solo tentativo sotto 5; un argomento forte ha media almeno 8. Le medie nel report sono arrotondate a una cifra decimale.

19. Errori, consistenza e recupero

La consistenza dei dati deve essere considerata oltre i confini del database relazionale. Una transazione SQLite può rendere indivisibili più scritture SQL, ma non comprende automaticamente la cancellazione di un file o l'aggiornamento di Chroma. Il programma adotta quindi operazioni di compensazione: conserva informazioni sufficienti a tentare il ripristino quando uno dei passaggi fallisce.

Un ripristino tentato non equivale a un'atomicità distribuita garantita. Per questo le procedure distinguono gli errori dell'operazione principale da quelli della pulizia o del recupero, e prevedono verifiche successive. La matrice degli errori collega le condizioni osservabili alle azioni diagnostiche, mentre il backup consente il recupero da situazioni che non possono essere risolte attraverso il normale flusso dell'interfaccia.

19.1 Confini transazionali

SQLite garantisce atomicità soltanto per le proprie transazioni. File, Chroma e report sono archivi distinti. Per cancellare una fonte, `materials.delete_material()` sposta il PDF in un file temporaneo, conserva i record Chroma da ripristinare, cancella indice e record SQL e conferma la transazione. Su errore tenta di ricreare i chunk e rimettere il PDF al suo posto. Per eliminare un esame, `exam_deletion.delete_exam()` sposta temporaneamente PDF e report, conserva una copia dei dati della collection, la elimina e conferma la transazione nel DB; su errore tenta di ripristinare collection e file. Errori specifici distinguono cancellazione riuscita con pulizia finale incompleta da cancellazione fallita con recupero incompleto.

Il reindex in `frontend_extras.py` opera **una fonte alla volta**: conserva vecchi documenti/metadati, cancella i vecchi chunk, richiama l'ingestor e aggiorna conteggi/hash nel DB. Se una fonte fallisce, tenta di ripristinarne i record Chroma precedenti. Non offre atomicità sull'intero insieme dei materiali dell'esame. Dopo un crash o un'operazione manuale, `verify_exam_material()` è il controllo centrale per individuare file mancanti, hash cambiati, differenze di conteggio e chunk orfani.

19.2 Matrice di errori osservabili

Condizione	Esito tipico	Diagnosi
Token assente o scaduto	API 401, UI verso login.	Rifare login; controllare cookie o bearer.
ID di altro studente	404.	Verificare proprietario di esame/sessione.
POST web con `Origin` estraneo	403.	Controllare host, schema e proxy.
PDF oltre limite	413.	Confrontare dimensione con `MAX_UPLOAD_BYTES`.
Firma o contenuto PDF non valido	400/422 secondo la fase.	Riesportare il file originale.
OCR fallito o nessun testo utile	Upload rifiutato.	Controllare leggibilità delle pagine.
File, hash o chunk incoerenti	Verifica `ok=false`; operazioni dipendenti possono dare 409.	Verifica e reindex dal PDF conservato.

Condizione	Esito tipico	Diagnosi
Quiz con meno domande fondate del richiesto	422 e quiz non accettato.	Ridurre numero o migliorare fonti.
Stato tutor incompatibile	409.	Ricaricare il dettaglio della sessione.
Quota provider	429 dove mappato o errore pubblico con attesa.	Controllare provider, chiave e retry indicato.
Report mancante	Download 404.	Rigenerare dal log se c'è un voto finale.

I messaggi pubblici passano da `public_error()` o dalle traduzioni nei router; il log server contiene la diagnostica. La diagnosi di un guasto richiede la lettura del traceback **completo**, comprese le fasi successive alla risposta LLM e precedenti alla persistenza. In una console Windows con encoding limitato, anche un carattere Unicode scritto su stdout può interrompere una richiesta: il guasto può quindi dipendere dalla scrittura del log anziché dalla generazione del contenuto. `X-Request-ID` aiuta a collegare richiesta, retrieval e chiamate al provider nei log.

19.3 Backup e ripristino

Un backup coerente comprende `data/study_coach.db`, `data/uploads/`, `data/chroma/` e `reports/`, raccolti quando non ci sono scritture in corso o tramite una procedura che garantisca coerenza. Per riprodurre l'ambiente conservare anche configurazione e prompt versionati; custodire `.env` separatamente. Dopo il ripristino: avviare il server, verificare login e lista esami, usare la verifica materiali su ogni esame, aprire una sessione storica, scaricare un report e provare retrieval su una fonte campione. `/health` da solo non dimostra l'integrità dei dati.

19.4 Diagnosi della configurazione LLM

`config/app.toml` sceglie attualmente Groq per il tutor, Mistral per il quiz con Gemini come fallback e Gemini per gli appunti con `qwen/qwen3.8-27b` su Groq come riserva. Ogni funzionalità ha una coppia provider/modello indipendente. Cambiare `GROQ_MODEL` non sostituisce un `TUTOR_MODEL` esplicito. `make_model()` richiede la chiave del provider scelto, applica il timeout e configura l'eventuale modalità JSON. `llm_errors.py` classifica limiti e problemi riprovabili e applica un numero finito di tentativi. La diagnosi di una funzione non disponibile richiede il controllo della **sua** coppia provider/modello e della relativa chiave: la presenza di una chiave generica non è sufficiente.

Per gli appunti, la sequenza dei log va letta per intero. Un errore Gemini seguito da una risposta Qwen validata dimostra che il fallback ha prodotto una parte; non dimostra che il revisore abbia terminato né che tutte le fonti siano state elaborate. La diagnosi dell'eventuale fallimento richiede gli eventi successivi, fino alla risposta HTTP finale. Le quote di Groq e Gemini sono esterne all'applicazione e possono cambiare nel tempo.

20. Qualità, verifica e limiti delle evidenze

La qualità del progetto va valutata specificando quale proprietà è stata verificata e con quali strumenti. I test deterministici dimostrano regole e comportamenti nelle condizioni preparate; le prove con un modello reale misurano un'esecuzione dipendente da prompt, materiale, provider e data. La valutazione completa deve mettere in relazione queste evidenze senza attribuire a una prova un ambito più ampio di quello effettivamente coperto.

La relazione conserva questa distinzione anche per i risultati storici. Un file di valutazione documenta un'esecuzione già avvenuta; non costituisce una nuova prova eseguita durante la preparazione del documento. Analogamente, controllare il testo di un PDF generato verifica la presenza di contenuti, mentre la leggibilità della pagina richiede anche un controllo visivo.

20.1 Test automatici

`test/test_*.py` copre schemi, autenticazione, cookie, proprietà API, configurazione, ingestione, controlli di qualità, routing dei modelli, quiz, tutor, adaptive engine, report, timer, appunti, interfaccia e recupero. Alcuni test sostituiscono provider, embedding o indice con fake/monkeypatch per rendere l'esecuzione deterministica: verificano invarianti come filtri, conteggi e rollback, ma non misurano la qualità semantica del modello reale. I test marcati `real_llm` sono esclusi da pytest per impostazione predefinita e richiedono credenziali/quota.

Il test del renderer delle tabelle crea un PDF con una tabella abbastanza lunga da occupare più pagine. Verifica che la prima e l'ultima riga siano presenti nel testo estraibile, che l'intestazione compaia in ogni pagina e che il separatore Markdown non sia stampato come contenuto. La verifica visiva resta necessaria per giudicare spazi, allineamenti e leggibilità: l'estrazione testuale, da sola, non individua una cella tagliata o una sovrapposizione.

20.2 Validazione real LLM

`eval/cases/` contiene casi con riferimento, input, atteso, ottenuto, esito e commento. La modalità di sola validazione di `eval/run_eval.py` verifica schema, presenza dei materiali e hash SHA-256; richiede almeno 24 casi. L'esecuzione con il modello richiede `GROQ_API_KEY` e conserva output JSON e riepilogo Markdown in `eval/results/` con data, modello, versione del prompt, ambiente e risposte grezze. Calcola conteggi, corrispondenza dei voti, errore medio assoluto e citazioni. Il runner fornisce al modello **tutto il testo di riferimento direttamente nel prompt**: non attraversa OCR, chunking, Chroma e ranking dell'app. I risultati storici descrivono le esecuzioni in cui sono stati prodotti.

20.3 Criteri di verifica

La verifica delle modifiche comprende test mirati al flusso interessato, regressione completa, controllo delle dipendenze bloccate e confronto con i casi di validazione. Una prova con modello reale consuma richieste del provider. Per confrontare esecuzioni diverse occorre conservare versione del codice, data, modello, prompt, hash degli input e output grezzi. La valutazione end to end richiede inoltre PDF reali, casi positivi e negativi con più fonti, controllo delle citazioni e giudizio umano su domande, voti e appunti. Le istruzioni operative per avviare il progetto e l'ambiente di test sono in AVVIO.md.

20.4 Ambito della documentazione

Le sezioni precedenti descrivono le funzioni implementate e i loro confini. Accuratezza accademica, disponibilità dei provider e funzionamento in una distribuzione pubblica multiistanza richiedono verifiche specifiche. La qualità effettiva va misurata con input riproducibili, riferimento atteso, risultato ottenuto e ripetizioni nel tempo.

21. Divisione dei contributi: agenti applicativi

La ripartizione dei contributi relativi ai sei moduli principali della cartella `backend/agents/` segue quanto riportato nel documento di progetto del 10 settembre 2026. La tabella associa ciascun modulo al relativo sviluppatore e ne descrive il ruolo nel funzionamento dell'applicazione. Si tratta dell'attribuzione dei contributi di sviluppo: i sei moduli non rappresentano necessariamente sei agenti linguistici eseguiti contemporaneamente. Alcuni realizzano elaborazioni documentali o regole deterministiche, mentre altri coordinano chiamate ai modelli.

Agente o modulo	Ruolo nell'applicazione	Sviluppatore
`ingestor.py`	Estrazione del testo dai PDF, OCR locale quando necessario, chunking configurabile, embedding locale con sentence-transformers e indicizzazione dei passaggi in ChromaDB per il recupero RAG.	Diabi Mohamed Zakaria
`quiz_generator.py`	Preparazione dei contesti a partire dai passaggi indicizzati nelle fonti selezionate e generazione del numero richiesto di domande aperte su tre livelli di difficoltà, con controlli di struttura, citazioni e duplicati.	Gouaiche Kamal
`socratic_tutor.py`	Classi SocraticTutor e DoubtTutor: gestione della sequenza domanda, risposta, domanda guida e valutazione; controlli di coerenza e severità del giudizio; gestione della contestazione del voto.	Diabi Mohamed Zakaria
`adaptive_engine.py`	Tracciamento dei punteggi per argomento e selezione della domanda successiva, dando priorità agli argomenti deboli e usando gli esiti affidabili per orientare il percorso.	Gouaiche Kamal
`notes_generator.py`	Generazione di appunti in Markdown e PDF dal materiale selezionato o dal log di una sessione. Il percorso basato sui materiali comprende suddivisione del contesto, coordinamento della scrittura, revisione della fedeltà e composizione del documento.	Diabi Mohamed Zakaria
`report_generator.py`	Generazione del report di fine sessione in Markdown e PDF, con media dei punteggi, dettaglio degli argomenti deboli e forti e sintesi conclusiva dei risultati.	Gouaiche Kamal

21.1 Riepilogo per sviluppatore

Diabi Mohamed Zakaria ha contribuito ai moduli `ingestor.py`, `socratic_tutor.py` e `notes_generator.py`. I tre contributi riguardano l'acquisizione e l'indicizzazione dei documenti, l'interazione didattica con lo studente e la produzione degli appunti. Queste funzioni collegano il materiale di partenza alle attività di esercitazione e ripasso.

Gouaiche Kamal ha contribuito ai moduli `quiz_generator.py`, `adaptive_engine.py` e `report_generator.py`. I tre contributi riguardano la costruzione del quiz, la scelta delle domande in base ai risultati e la restituzione del bilancio della sessione. Queste funzioni organizzano il percorso di verifica e ne rendono consultabili gli esiti.

22. Utilizzo di intelligenza artificiale durante lo sviluppo

Nel corso del progetto sono stati utilizzati strumenti di intelligenza artificiale come supporto alla progettazione iniziale e ad alcune attività di sviluppo. Questo impiego riguarda il processo di realizzazione del software ed è distinto dall'utilizzo dei modelli linguistici integrati in StudyCoach per quiz, tutor e appunti, descritto nei capitoli precedenti.

22.1 GitHub Copilot

GitHub Copilot, con piano Student, è stato utilizzato con selezione automatica del modello, indicata come autoselect. Il supporto ha riguardato principalmente la finalizzazione della parte grafica dell'interfaccia e l'individuazione di bug ed errori nel codice durante lo sviluppo. L'interfaccia attuale è realizzata mediante FastAPI e template Jinja, con HTML, CSS e JavaScript per la presentazione e le interazioni nel browser.

22.2 Claude

Claude, tramite chat, è stato utilizzato nella fase iniziale, prima dell'avvio della scrittura del codice, come supporto per definire un'idea generale dell'applicazione. Il confronto ha aiutato a ragionare sulla suddivisione delle responsabilità fra interfaccia, API REST e livello degli agenti applicativi, oltre a comprendere l'impiego del framework Agno, non utilizzato in precedenza dal gruppo.

Il riferimento alla fase iniziale riguarda il processo di impostazione del progetto. La descrizione tecnica della presente relazione segue invece l'assetto attuale: FastAPI gestisce l'applicazione e le API, Jinja rende le pagine dell'interfaccia e i moduli del backend realizzano le funzioni documentali e didattiche.